Rapport Projet RL - TP2 AI Driver Unity (PPO)

Ajax DESHAYES--HUET

Introduction

Contexte

Ce TP2 porte sur l'entrainement d'un agent de conduite autonome dans Unity avec ML-Agents, en utilisant l'algorithme PPO. L'agent doit apprendre a suivre la piste de la scene track1 de facon fiable, avec peu de crashes, tout en conservant une progression stable pendant l'entrainement.

Objectifs

Ce rapport a pour objectifs de:

1. presenter la configuration de depart et les hyperparametres modifies,
2. comparer les resultats de trois experiences (E1, E2, E3),
3. analyser le compromis entre performance, stabilite et robustesse en inference,
4. justifier la configuration finale retenue.

Methodologie

Parametres modifiables

- Batch size.
- Buffer size.
- Learning rate.
- Epsilon.
- Beta.
- Lambda.

- Num epoch.
- Gamma.

Logique generale du rapport

Le rapport suit une demarche iterative: chaque experience sert a observer un comportement, identifier une limite, puis ajuster les parametres.
Les resultats de chaque run orientent directement l'objectif de l'experience suivante.

Fonction de reward utilisee (non modifiee)

Dans ce TP2, je n'ai pas modifie la fonction de reward entre E1, E2 et E3. Le but etait d'isoler l'effet des hyperparametres PPO (batch size, learning rate, epsilon, beta) sans changer l'objectif d'apprentissage.

Valeurs de reward utilisees dans le script agent (car_agent_template_track1.cs):

- Progression vers la cible: `AddReward(+0.00001)` .
- Eloignement de la cible: `AddReward(-0.001)` .
- Collision avec un objet tagge `Death` : `SetReward(-1.0)` puis fin d'episode.
- Atteinte d'une cible taggee `Target` : `SetReward(+1.0)` puis passage a la cible suivante.

Ce choix rend la comparaison des trois experiences plus robuste: les differences observees viennent des hyperparametres PPO, pas d'un reward shaping different.

Experiences

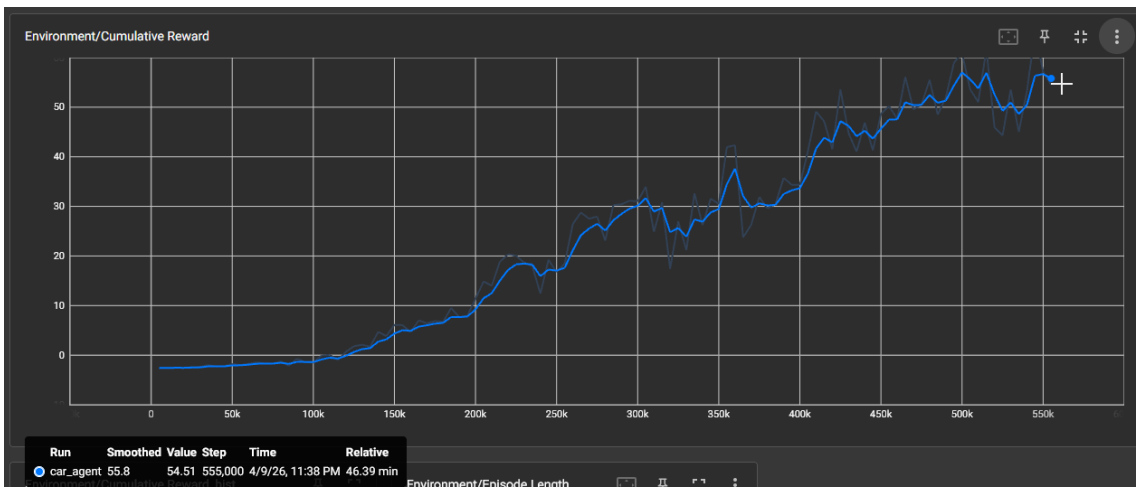
Tableau rapide

Experience	Batch size	lr	Epsilon	Beta	Lambda	Num epoch	Attente principale
E1	512	3e-4	0.2	0.005	0.95	3	Baseline
E2	1024	1.5e-4	0.15	0.003	0.95	3	Reward finale plus elevee mais moins stable

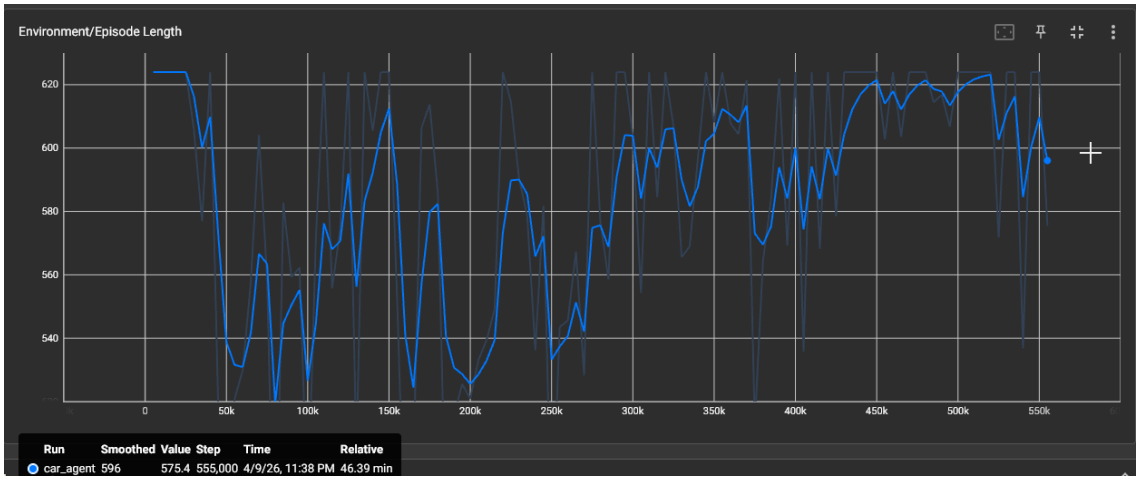
Experience	Batch size	lr	Epsilon	Beta	Lambda	Num epoch	Attente principale
E3	768	2.2e-4	0.17	0.004	0.95	3	Compromis stabilite-rapidite: apprentissage rapide + inference robuste

Experience E1

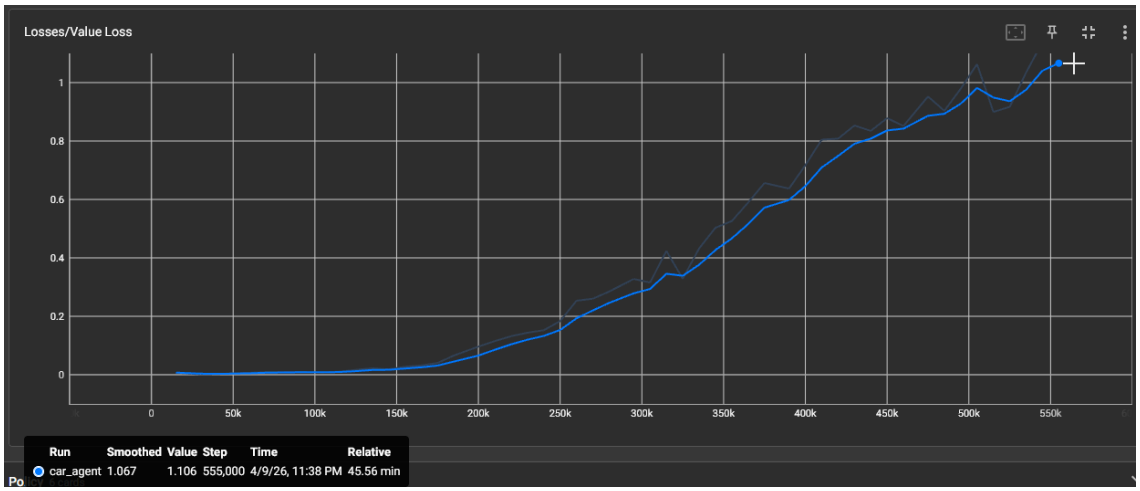
- **Choix des parametres**: batch size 512, learning rate 3e-4, epsilon 0.2, beta 0.005, lambda 0.95, num epoch 3.
- **Ce qu'on teste**: la baseline PPO sur track1.
- **Resultats observes**:
  - Run lance avec `--run-id=Experience1` .



- La courbe `Environment/Cumulative Reward` monte globalement tout le long du run. - D'apres l'export JSON, la valeur passe d'environ `-2.57` au debut a un pic proche de `65.22` (step `545000`), puis termine vers `54.51` (step `555000`) apres un dip suivi d'une remontee. -



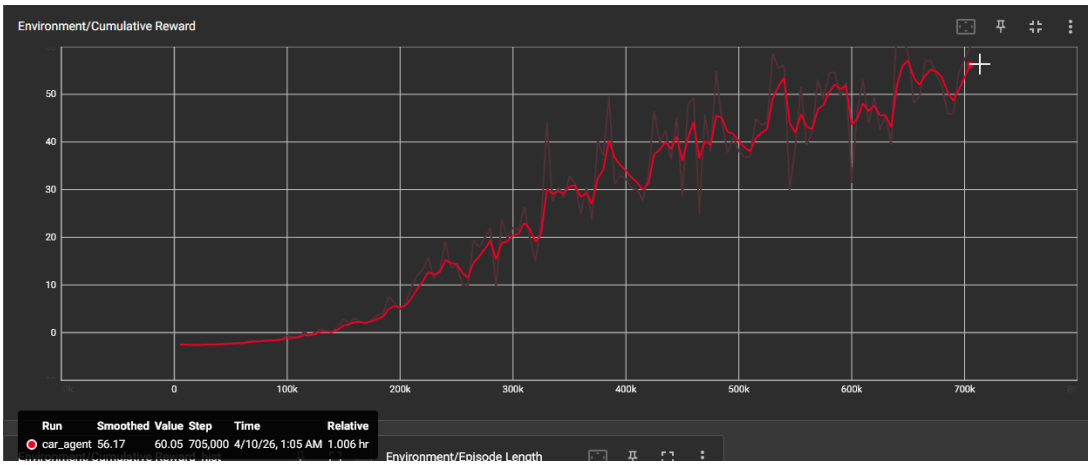
- `Environment/Episode Length` reste bruitee et souvent proche du plafond (`624`), avec quelques baisses ponctuelles (par exemple autour de `453` au step `80000`).



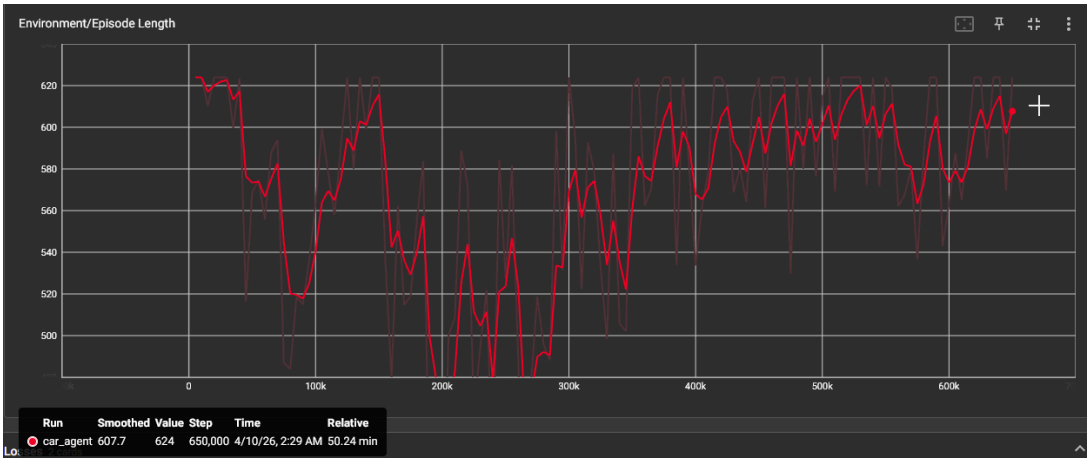
- `Losses/Policy Loss` oscille dans une plage assez stable, globalement entre `0.027` et `0.045`. - `Losses/Value Loss` augmente progressivement (d'environ `0.007` au debut jusqu'a ~`1.10` en fin de run), avec des fluctuations. - Test en inference apres entrainement: premier essai en echec (collision mur), deuxieme essai valide en `23.26 s` pour un tour. - **Interpretation**: sur ce premier run, j'ai constate une progression nette de la reward, mais une regularite encore inconstante. La longueur d'episode souvent elevee et le premier crash en inference montrent qu'il reste un manque de fiabilite. Le policy loss reste raisonnable, alors que la hausse du value loss suggere que le critic suit plus difficilement quand les retours augmentent. - **Duree du run**: environ `46 minutes`. - **Critere d'arret**: j'ai arrete manuellement apres un dip qui s'etait re-equilibre, avec une tendance reward encore globalement ascendante.

Experience E2

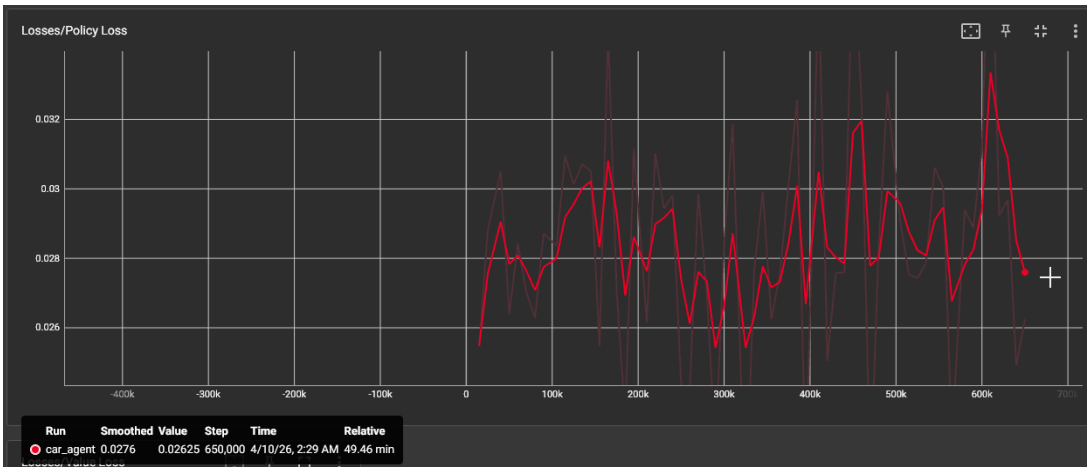
- **Parametres choisis et justification**:
  - `batch_size` : `512 -> 1024` pour lisser les gradients et limiter les comportements erratiques.
  - `learning_rate` : `3e-4 -> 1.5e-4` pour des updates moins agressives.
  - `epsilon` (PPO clip): `0.2 -> 0.15` pour un clipping plus prudent.
  - `beta` (entropy): `0.005 -> 0.003` pour reduire l'aleatoire en fin d'apprentissage.
  - `lambda` et `num_epoch` inchanges (`0.95` , `3`) pour garder une base comparable a E1.
- **Ce qu'on teste**: conserver la progression reward de E1 avec une conduite plus reguliere (moins de collisions, meilleur temps de tour).
- **Resultats observes**:
  - Run lance avec `--run-id=Experience2` .



- Cumulative reward: -2.53 (debut) → 60.05 (step 705k, **plus eleve que E1 final!**)



- Episode length: Similaire a E1, souvent proche du max 624 avec dips ponctuels.

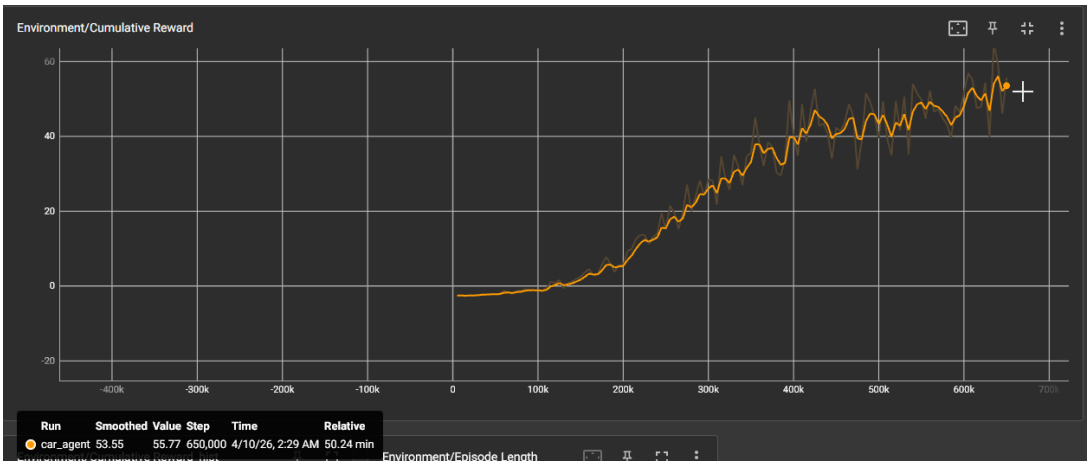


- Policy loss: Oscille entre 0.024-0.030 (stable, comparable a E1). - Value loss: Monte a 1.47 (legerement plus instabile que E1 qui atteignait max 1.10-1.25). - Tests en inference: plusieurs crashes et comportements erratiques (agent reste bloque apres avoir rate une reward, crash au debut d'autres parcours). Un essai a complete le tour en `20.18 sec` (plus rapide que E1).

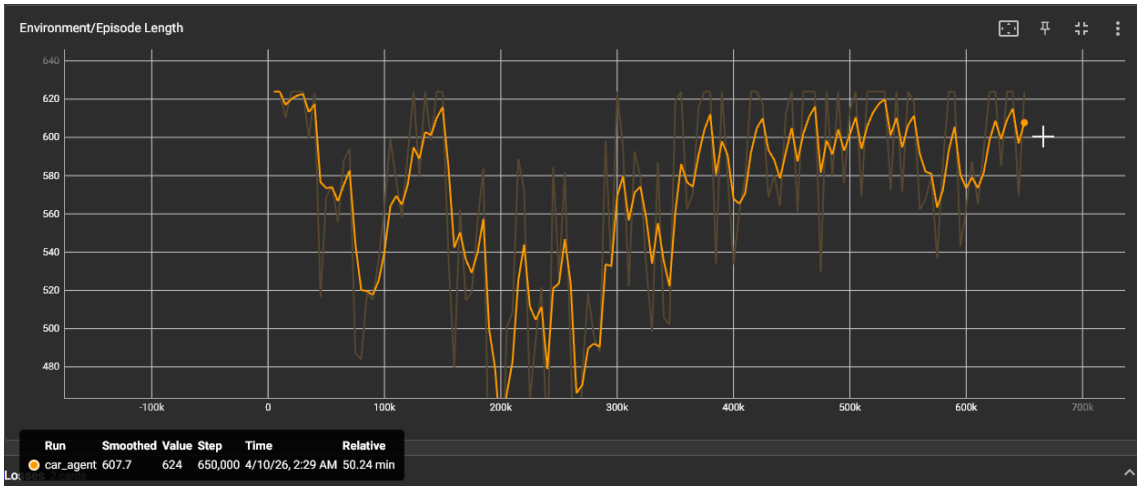
- **Interpretation**: E2 atteint une reward finale plus elevee que E1 (+5.54), avec un tour reussi plus rapide (20.18 s vs 23.26 s). En pratique, j'ai aussi observe plus de runs rates en inference (crashes et blocages), ce qui confirme la baisse de fiabilite. Le gain de performance brute est donc reel, mais avec une stabilite plus fragile.

Experience E3

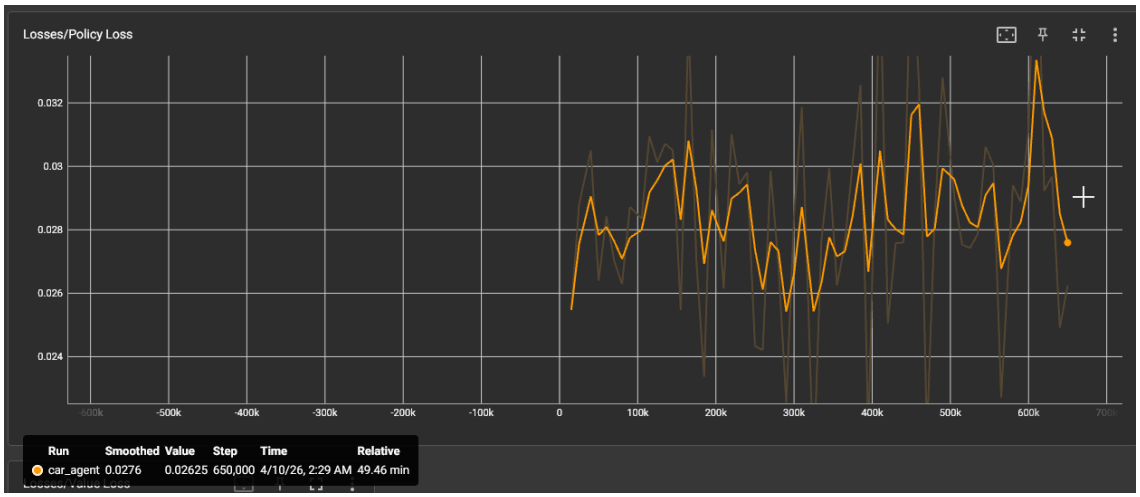
- **Parametres choisis et justification**:
  - `batch_size` : `E1=512, E2=1024 -> E3=768` pour un compromis de lissage entre E1 et E2.
  - `learning_rate` : `E1=3e-4, E2=1.5e-4 -> E3=2.2e-4` pour equilibrer vitesse d'apprentissage et stabilite.
  - `epsilon` (PPO clip): `E1=0.2, E2=0.15 -> E3=0.17` pour un clipping intermediaire.
  - `beta` (entropy): `E1=0.005, E2=0.003 -> E3=0.004` pour equilibrer exploration et regularite de conduite.
  - `lambda` et `num_epoch` inchanges (`0.95` , `3`) pour isoler l'effet des autres ajustements.
- **Ce qu'on teste**: un compromis entre E1 et E2, avec une reward finale elevee et une inference plus robuste.
- **Resultats observes**:
  - Run lance avec `--run-id=Experience3` .
  - Training duration: environ `50 minutes` pour atteindre 650k steps (`max_steps: 650000` arrêt automatique).

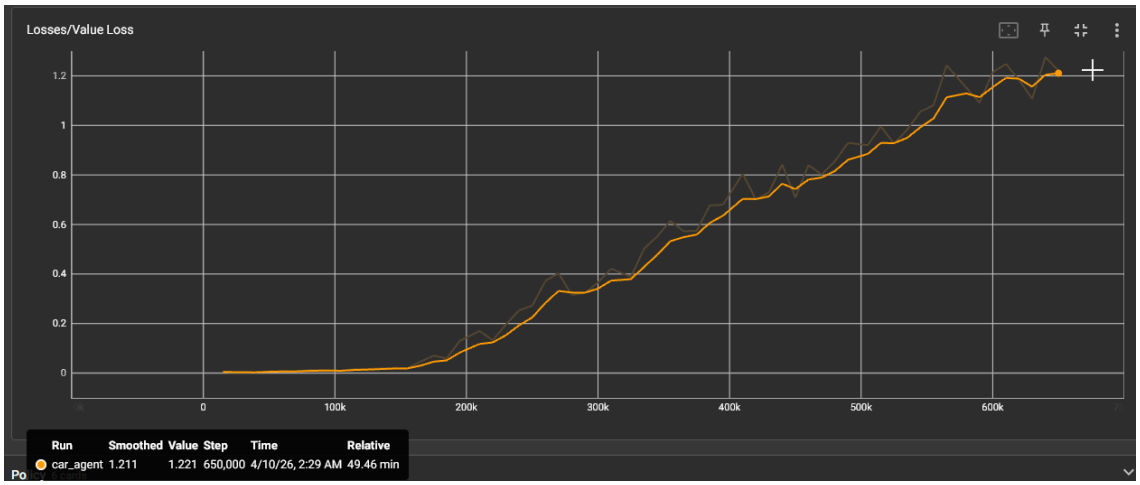


- Cumulative reward: -2.53 (debut) → 55.77 (step 650k)



- Episode length: Similaire a E1/E2, souvent proche du max 624 avec dips ponctuels.





- Policy loss: Oscille entre 0.025-0.027 (tres stable, comparable a E1 et E2). - Value loss: Monte a 1.22 (entre E1 ~1.10-1.25 et E2 ~1.47) → meilleur que E2! - Tests en inference: agent observe plus stable, rare de rater une reward ou de crash dans les virages serre. Temps tour mesure: `20.91 sec`. - **Interpretation**: E3 atteint bien le compromis vise: reward finale 55.77 (entre E1 et E2), value loss a 1.22 (meilleure que E2), et inference plus fiable que E2. Sur les essais que j'ai faits, la conduite paraissait plus propre dans les virages serres et moins sujette aux erreurs grossieres. Le temps de tour (20.91 s) reste proche de E2 tout en gardant une meilleure stabilite.

Resultats

Synthese courte: E1 est la baseline la plus stable, E2 atteint la meilleure reward finale mais avec plus d'instabilite, et E3 donne le meilleur compromis global entre performance, stabilite et inference.

Comparaison finale

Critere	E1	E2	E3
Reward finale	54.51	60.05	55.77
Value loss finale	1.10-1.25	1.47	1.22
Temps tour moyen (reussi)	23.26 s	20.18 s	20.91 s
Robustesse inference	Stable mais lente	Plus rapide mais moins fiable	Meilleur compromis

Analyse

Les resultats montrent un compromis net entre rapidite et stabilite: E2 maximise la reward, mais perd en robustesse d'inference. E3 fournit le meilleur equilibre avec une value loss mieux maitrisee et un comportement plus fiable; les limites restent le faible nombre de runs (1/config) et l'alea de l'environnement.

Conclusion

Je retiens E3 pour ce TP2: reward finale elevee, value loss mieux controlee que E2, et comportement en inference plus solide. Le point principal du projet reste le meme: accelerer l'apprentissage n'a d'interet que si la stabilite reste acceptable. Pour la suite, je testerais en priorite un learning rate decay ou une prolongation legere de E3.